Listing 8.3 `document_vector_database.py`

```
# code above omitted
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
vector_database = X.toarray()     #1

def cosine_similarity_search(query,
                             database,
                             vectorizer,
                             top_n=5):     #2
    query_vec = vectorizer.transform([query]).toarray()
    similarities = cosine_similarity(query_vec, database)[0]
    top_indices = np.argsort(-similarities)[:top_n]  # Top n indices
    return [(idx, similarities[idx]) for idx in top_indices]

while True:      #3
    query = input("Enter a search query (or 'exit' to stop): ")
    if query.lower() == 'exit':
        break
    top_n = int(input("How many top matches do you want to see? "))
    search_results = cosine_similarity_search(query,
                                              vector_database,
                                              vectorizer,
                                              top_n)

    print("Top Matched Documents:")
    for idx, score in search_results:
        print(f"- {documents[idx]} (Score: {score:.4f})")   #4

    print("\n")
###Output
Enter a search query (or 'exit' to stop): blue
How many top matches do you want to see? 3
Top Matched Documents:
- The sky is blue and beautiful. (Score: 0.4080)
- Love this blue and beautiful sky! (Score: 0.3439)
- The brown fox is quick and the blue dog is lazy! (Score: 0.2560)
```

#1 Stores the document vectors into an array
#2 The function to perform similarity matching on query returns, matches, and similarity scores
#3 The main input loop
#4 Loops through results and outputs text and similarity score

Run this exercise to see the output (F5 in VS Code). Enter any text you like, and see the results of documents being returned. This search form works well for matching words and phrases with similar words and phrases. This form of search misses the word context and meaning from the document. In the next section, we'll look at a way of transforming documents into vectors that better preserves their semantic meaning.